DFSha: Informe final

Hito 4 | ST0263 Tópicos Especiales en Telemática / SI3007 Sistemas Distribuidos, 2026-2 | Universidad EAFIT

Andrés Vélez Rendón Jose Luis Restrepo

24 de octubre de 2026

Índice general

1. Objetivo, problema y marco teórico **3**

Objetivo . . . 3

Problema . . . 3

Marco teórico . . . 3

2. Descripción del servicio **4**

Componentes . . . 4

Operaciones del cliente . . . 4

Cómo se cubren los requisitos no funcionales . . . 5

Fuera del alcance . . . 5

3. Arquitectura **5**

Plano de control y plano de datos . . . 5

Qué estado se replica y cuál no . . . 5

Organización del código . . . 6

4. Comunicaciones **7**

Protocolo por cada relación . . . 7

API del ControlNode para el cliente . . . 7

API interna del ControlNode . . . 9

API del DataNode . . . 9

Errores y reintentos . . . 9

Cómo funciona una subida . . . 10

Cómo funciona una descarga . . . 11

5. Algoritmos de particionamiento y distribución **11**

Tamaño de bloque . . . 11

Número de copias . . . 12

Ubicación de los bloques . . . 12

Identificación por contenido . . . 12

Rebalanceo . . . 12

6. Alta disponibilidad **13**

ControlNodes . . . 13

DataNodes . . . 13

7. Replicación y consistencia **13**

Datos: bloques inmutables . . . 13

Metadatos: un registro replicado . . . 14

Un archivo aparece completo o no aparece . . . 14

Mantenimiento sin riesgo para los datos . . . 14

Garantías . . . 14

8. Seguridad **15**

En la red . . . 15

En disco . . . 15

Usuarios y permisos . . . 15

Secretos . . . 15

9. Entorno de ejecución **15**

Docker Compose en una máquina . . . 15

Procesos nativos . . . 16

Varias máquinas en AWS Academy . . . 16

Parámetros . . . 16

10. Pruebas y análisis de resultados **17**

Pruebas automáticas . . . 17

Prueba de integración . . . 17

Pruebas con Docker y con cuatro VMs simuladas . . . 18

Rendimiento . . . 19

Resultados en AWS . . . 19

Hallazgos corregidos durante las pruebas . . . 19

11. Cambios respecto a los hitos anteriores **20**

12. Limitaciones **20**

Notas **21**

Referencias **22**

Opción asignada: Opción 1, Cliente/Servidor, con el servicio distribuido entre varios nodos dentro de una red privada.

Este es el informe técnico final del proyecto. Reúne lo que se especificó en el Hito 1 y se construyó en el Hito 2, y describe lo que se agregó para cubrir el Hito 3 (alta disponibilidad, replicación, consistencia y seguridad) y la entrega final. Como en los informes anteriores, las decisiones se enuncian en el cuerpo y se justifican en las **Notas** del final. Las notas de otros hitos se citan como *Hito 1, N3*.

Código fuente: `https://github.com/AndresVelezR/DFSha---Distributed-Systems`.

---

1. Objetivo, problema y marco teórico

Objetivo

Diseñar e implementar un sistema de archivos distribuido que guarde archivos grandes repartidos entre varios nodos, con escritura y lectura distribuidas, y que siga funcionando sin perder datos cuando falla una máquina. Todo con las comunicaciones y el almacenamiento cifrados, control de acceso por usuario y ubicación dinámica de los datos.

Problema

Un solo servidor de archivos es a la vez el límite de capacidad, el cuello de botella de la red y un punto único de falla: si se cae, no hay servicio, y si pierde su disco, se pierden los archivos. Repartir los archivos entre varias máquinas resuelve la capacidad y el rendimiento, pero trae tres problemas nuevos. Hay que saber en todo momento dónde está cada pedazo de cada archivo, y esa información no se puede perder. Hay que mantener varias copias sin que diverjan. Y hay que decidir qué hacer cuando una máquina deja de responder en la mitad de una operación.

Marco teórico

Un sistema de archivos distribuido guarda archivos repartidos en varias máquinas y los presenta como un solo árbol de directorios. Hay dos enfoques. En el almacenamiento por objetos, como S3, la unidad es el archivo completo. En el almacenamiento por bloques, como GFS y HDFS, el archivo se parte y cada parte se ubica por separado. DFSha es por bloques porque el enunciado pide que cada archivo quede repartido entre varios nodos y que haya concurrencia dentro de un mismo archivo (Hito 1, N1).

De GFS y HDFS se toman dos ideas. La primera es separar el plano de control del plano de datos: un nodo coordinador guarda solo metadatos y decide dónde va cada bloque, y los bytes viajan directamente entre el cliente y los nodos de almacenamiento. La segunda es la escritura única: un bloque guardado no se modifica nunca, así que dos copias del mismo bloque son idénticas por construcción.

El coordinador no puede ser una sola máquina, porque sin él nadie sabe dónde está nada. Replicarlo exige que todas sus copias apliquen los mismos cambios en el mismo orden, aunque algunas se caigan o se desconecten. Ese es el problema del **consenso**, y DFSha lo resuelve con **Raft [N1]**. En Raft, las réplicas eligen un líder. El

líder recibe cada cambio, lo agrega a un registro y lo replica a las demás. Un cambio se considera confirmado cuando una mayoría lo guardó en disco. Con tres réplicas la mayoría es dos, así que el sistema tolera la caída de una cualquiera, y nunca puede haber dos líderes aceptando cambios a la vez, porque cada uno necesitaría su propia mayoría.

Para la seguridad se usan piezas estándar: TLS para cifrar las comunicaciones, AES-256-GCM para cifrar y autenticar los bloques en disco, bcrypt para guardar contraseñas y HMAC-SHA256 para firmar los permisos que emite el sistema.

---

2. Descripción del servicio

DFSha guarda archivos de cualquier tipo partiéndolos en bloques que se reparten y se replican entre servidores, y los recupera reconstruyéndolos a partir de esos bloques. Un archivo grande nunca se guarda completo en una sola máquina ni se carga completo en memoria.

Componentes

- **Cliente** (Python, sin dependencias externas). Programa de línea de comandos que también se puede usar como biblioteca. Se autentica, ejecuta los comandos, parte los archivos al subirlos y los reconstruye al bajarlos. No es parte del servicio.
- **ControlNode** (Go). Hay tres. Guardan los metadatos: el árbol de directorios de cada usuario, qué bloques forman cada archivo y dónde está cada copia. Uno de ellos es el líder: atiende al cliente, calcula los planes de subida y bajada, y repara y rebalancea las copias. No guarda datos de archivos.
- **DataNode** (Go). Tres o más. Guardan los bloques cifrados en su disco, los entregan completos o por rangos y los copian a otros DataNodes cuando el líder lo ordena.

Operaciones del cliente

Requisito	Comandos	Qué hacen
RF1, gestión	`ls`, `cd`, `pwd`, `mkdir`, `rmdir`, `rm`, `mv`, `stat`	Recorren y cambian el árbol del usuario. Solo tocan metadatos. `stat` muestra tamaño, bloques y en qué DataNodes está cada copia
RF2, transferencia	`put` o `send`, `get` o `receive`	Suben y bajan un archivo completo, con varios bloques en paralelo
RF3, acceso	`open`, `lock`, `read`, `write`, `close`	Abren un archivo y devuelven un identificador (*handle*); leen un rango de bytes descargando solo los bloques que lo contienen; reservan el archivo para un solo escritor; escriben bytes desde una posición publicando una versión nueva
Clúster	`status`	Muestra los DataNodes, su ocupación y el estado de Raft

Cómo se cubren los requisitos no funcionales

Requisito	Cómo se cubre
RNF1, escalabilidad	Los DataNodes se agregan con el sistema en marcha: un nodo nuevo solo arranca y se registra, y empieza a recibir bloques. El tamaño de bloque se adapta al número de nodos y los ControlNodes solo guardan metadatos (secciones 5 y 9)
RNF2, disponibilidad	Tres ControlNodes con Raft toleran la caída de uno. Cada bloque tiene dos copias en DataNodes distintos, y el líder repone las que se pierden (sección 6)
RNF3, consistencia	Bloques inmutables identificados por su SHA-256, metadatos replicados por mayoría, un solo escritor por archivo y publicación atómica (sección 7)
RNF4, particionamiento	Todo archivo se parte en bloques repartidos entre DataNodes, y las copias de un mismo bloque nunca quedan en el mismo nodo (sección 5)
RNF5, rendimiento	El cliente transfiere varios bloques a la vez contra DataNodes distintos, y cada DataNode atiende varias peticiones en paralelo (secciones 5 y 10)
RNF6, seguridad	HTTPS en todos los canales y TLS mutuo entre ControlNodes, bloques cifrados en disco, usuarios con contraseña, árbol privado por usuario y permisos firmados para cada acceso a un bloque (sección 8)
RNF7, transparencia	El cliente no conoce ningún DataNode de antemano: recibe sus direcciones en cada plan, y si un ControlNode no es el líder, le indica cuál es. El usuario solo ve rutas

Fuera del alcance

No se monta como unidad del sistema operativo: el acceso es por línea de comandos y API. No hay permisos compartidos entre usuarios ni grupos: cada usuario ve solo su árbol **[N9]**. El número de ControlNodes es fijo en tres.

---

3. Arquitectura

Tipo de arquitectura: Maestro y Trabajador, con el maestro replicado. Los ControlNodes coordinan y saben dónde está todo, y los DataNodes guardan los datos. A diferencia del Hito 2, el maestro ya no es una sola máquina sino un grupo de tres que se mantiene igual con Raft.

Plano de control y plano de datos

El cliente le pregunta al líder qué hacer y le responde un **plan**: para cada bloque, en qué DataNodes va o de cuáles se lee, con un permiso firmado para cada uno. Con ese plan, el cliente mueve los bloques directamente con los DataNodes. El líder nunca recibe el contenido de un archivo, así que el tamaño de los archivos no lo convierte en cuello de botella.

Qué estado se replica y cuál no

Los ControlNodes guardan dos tipos de estado, y la diferencia es central en el diseño **[N2]**:

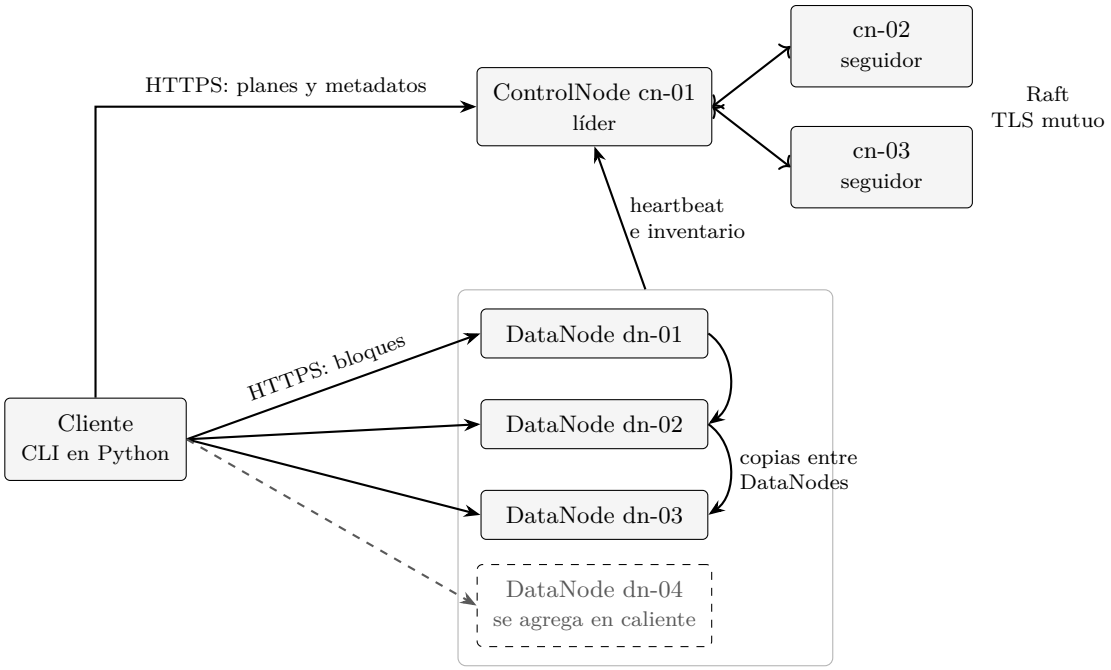


Figura 1: Componentes y comunicaciones. Los bloques nunca pasan por los ControlNodes.

- **Replicado por Raft**: directorios, archivos publicados con su lista de bloques, reservas de subida, handles abiertos y subidas ya confirmadas. Es lo que no se puede perder ni contradecir: si se perdiera, se perderían los archivos aunque sus bloques siguieran en disco. Vive en el registro de Raft en disco y es idéntico en los tres ControlNodes.
- **Memoria del nodo**: qué DataNodes están vivos, cuánto ocupa cada uno y qué bloques tiene. Es información que los propios DataNodes vuelven a enviar cada pocos segundos, así que si se pierde se reconstruye sola. Los DataNodes mandan su heartbeat a los tres ControlNodes, de modo que un seguidor que gana una elección ya sabe qué DataNodes están vivos.

Organización del código

Cada nodo es un binario independiente. Dentro de cada uno el código se separa en archivos por responsabilidad, como en el Hito 2.

Archivo	Responsabilidad
`controlnode/consensus.go`	Raft: registro en disco, snapshots, TLS mutuo y la máquina de estados que aplica cada cambio confirmado
`controlnode/server.go`	Estado del servidor, rutas, comprobación de liderazgo y publicación de cambios por Raft
`controlnode/planner.go`	Tamaño de bloque, número de copias y ubicación. Funciones puras, sin HTTP
`controlnode/handlers_*.go`	Endpoints por tema: autenticación, sistema de archivos, subida, descarga y clúster
`controlnode/maintenance.go`	Reparación, rebalanceo y borrado de bloques sin uso
`datanode/storage.go`	Disco: cifrado, verificación, escritura atómica e inventario
`datanode/handlers.go`	Endpoints de bloques y validación de permisos

Archivo	Responsabilidad
`datanode/control_client.go`, `peer_client.go`	Registro y heartbeat hacia los ControlNodes; copias hacia otros DataNodes
`datanode/ownership.go`	Registro de qué cuenta subió cada bloque
`internal/common`	TLS, secretos y permisos firmados, compartidos por ambos nodos
`admin`	Genera la CA, los certificados, las claves y los usuarios
`client/dfsha.py`	Cliente de línea de comandos y API

---

4. Comunicaciones

El control viaja en JSON y los bloques como cuerpo binario sin codificar, todo sobre HTTPS (Hito 1, N11). La única excepción es Raft, que usa su propio protocolo binario sobre TCP con TLS mutuo. Cada ControlNode expone su API en el puerto **8000** y Raft en el **9000**; cada DataNode expone la suya en el **8001**.

Protocolo por cada relación

Relación	Expone	Protocolo	Autenticación	Para qué
Cliente y ControlNode	ControlNode	REST/JSON sobre HTTPS	Token de sesión	Login, sistema de archivos, planes, confirmación y estado
Cliente y DataNode	DataNode	REST/binario sobre HTTPS	Permiso firmado por bloque y nodo	Subir y bajar bloques, completos o por rango
DataNode y DataNode	DataNode	REST/binario sobre HTTPS	Permiso del cliente, o clave del clúster en reparaciones	Propagar la copia al subir y reponer copias
ControlNode y DataNode	ambos	REST/JSON sobre HTTPS	Clave del clúster	Registro, heartbeat e inventario; órdenes de copiar, verificar y borrar
ControlNode y ControlNode	ControlNode	Raft sobre TCP con TLS mutuo	Certificado del clúster en ambos extremos	Elección de líder y réplica del registro de metadatos

API del ControlNode para el cliente

Todas, salvo `login` y `health`, requieren `Authorization: Bearer` con el token de sesión. Si el nodo que recibe la petición no es el líder, responde `503` con la URL del líder en el campo `leader`, y el cliente repite la petición allá.

Operación	Solicitud	Respuesta	Errores
`POST /auth/login`	`{username, password}`	`200 {token, expires_in}`	`401`

Operación	Solicitud	Respuesta	Errores
`GET /fs/ls?path=`	sin cuerpo	`200 {entries:[{name, type, size, blocks, replicas_ok}]}`	`404`
`POST /fs/mkdir`	`{path}`	`201`	`404` sin padre, `409` ya existe
`DELETE /fs/rmdir?path=`	sin cuerpo	`204`	`404`, `409` no vacío
`DELETE /fs/rm?path=`	sin cuerpo	`204`	`404`, `409` con escritor
`POST /fs/mv`	`{src, dst}`	`200`	`404`, `409` destino existe
`GET /fs/stat?path=`	sin cuerpo	`200 {size, sha256, block_size, blocks:[{index, block_id, size, stored_on}], replicas_ok}`	`404`
`POST /files/upload/plan`	`{path, size, sha256}`	`200` plan de escritura	`404`, `409` reservado, `503` sin DataNodes, `507` no cabe
`POST /files/upload/renew`	`{upload_id}`	`200` plan con permisos nuevos	`410` vencida
`POST /files/upload/commit`	`{upload_id, sha256, blocks:[{index, block_id, size}]}`	`200 {status:"publicado"}`	`409` bloque sin copias, `410` vencida
`POST /files/upload/abort`	`{upload_id}`	`204`	`403`
`GET /files/download/plan?path=`	sin cuerpo	`200` plan de lectura	`404`, `503` bloque sin copias vivas
`POST /files/open`, `/files/lock`	`{path, mode}`	`200 {handle, expires_at}`	`404`, `409` ya hay escritor
`POST /files/renew`, `/files/close`	`{handle}`	`200`, `204`	`403`, `410`
`GET /cluster/status`	sin cuerpo	`200 {nodes, files, raft}`	
`GET /health`	sin autenticación	`200 {node_id, role, leader, alive_datanodes}`	

Un plan de escritura de un archivo de 100 MiB en el clúster de tres DataNodes; el identificador y los permisos están abreviados:

```
POST /files/upload/plan  { "path": "/prueba/original.bin", "size": 104857600, "sha256": "24cd..." }
200 OK
{ "upload_id": "9c1e...", "block_size": 8388608, "n_blocks": 13,
  "expires_at": "2026-10-25T04:30:12Z",
  "blocks": [
    { "index": 0, "targets": [ {"node":"dn-02","url":"https://172.31.0.12:8001","token":"eyJ..."},
                               {"node":"dn-03","url":"https://172.31.0.13:8001","token":"eyJ..."} ] },
    ... ] }
```

API interna del ControlNode

Operación	Quién la usa	Para qué
`POST /cluster/register`	DataNode → ControlNode	Alta al arrancar con `{node_id, url, capacity, used}`
`POST /cluster/heartbeat`	DataNode → ControlNode	Señal de vida cada 3 s con los mismos datos. Se envía a los tres ControlNodes
`POST /cluster/blockreport`	DataNode → ControlNode	Lista de bloques que tiene el DataNode, cada 30 s

Todas exigen la clave del clúster en la cabecera `X-Cluster-Key`. Los endpoints `/cluster/replicate` y `/cluster/election` que definió el Hito 2 ya no existen: esa comunicación es ahora Raft, en el puerto 9000 [**N1**].

API del DataNode

Operación	La usa	Solicitud	Respuesta	Errores
`PUT /blocks/{id}`	Cliente, DataNode	Cuerpo binario y permiso de escritura. `X-Forward-To` con los otros destinos y sus permisos	`201` si lo creó, `200` si ya existía; `{block_id, size, stored_on}`	`401`, `409` firma no coincide, `507`
`GET /blocks/{id}`	Cliente	Permiso de lectura y opcionalmente `Range`	`200` o `206`, binario	`401`, `404`, `500` corrupto
`HEAD /blocks/{id}`	ControlNode	Clave del clúster y opcionalmente `X-Block-Owner`	`200` con firma y tamaño	`404`
`POST /blocks/{id}/replicate`	ControlNode	`{target:{node, url}}`	`200`, el bloque quedó en el destino	`404`, `502`
`DELETE /blocks/{id}`	ControlNode	Clave del clúster	`204`	`401`
`GET /inventory`	ControlNode	Clave del clúster	`{blocks, used, capacity}`	`401`
`GET /health`	Operación	sin autenticación	`200 {node_id}`	

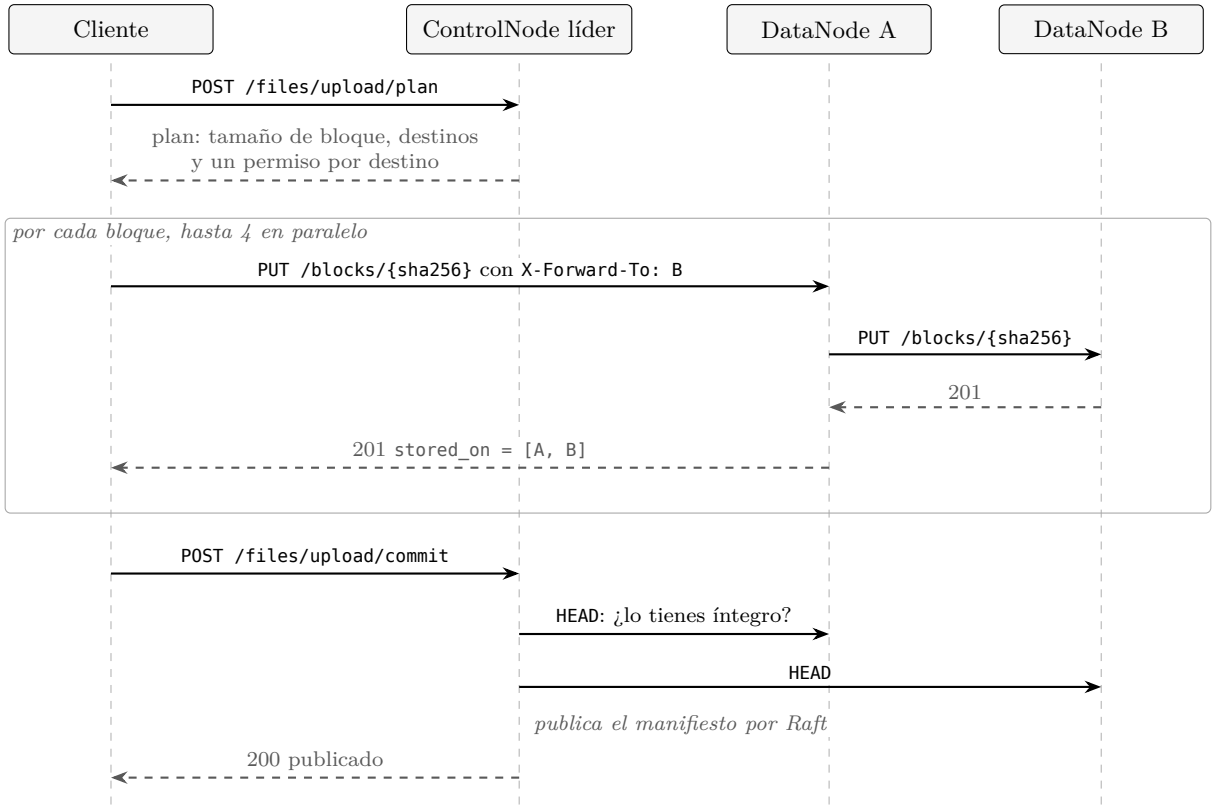
El identificador `id` de un bloque es el SHA-256 de su contenido en hexadecimal.

Errores y reintentos

Situación	Comportamiento
El ControlNode contactado no es el líder	Responde `503` con la URL del líder y el cliente repite allá
No hay líder, durante una elección	El cliente recorre los ControlNodes y vuelve a intentar a los 1, 2 y 4 s. Una elección tarda entre 2 y 4 s

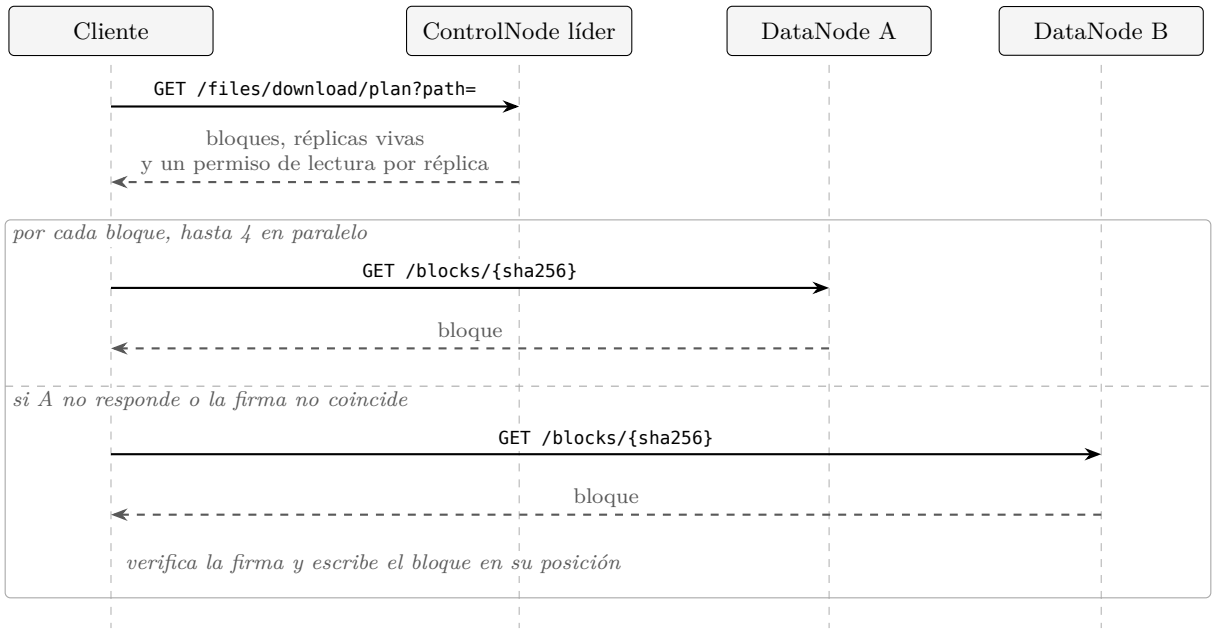
Situación	Comportamiento
Fallo de red al subir un bloque	Hasta 3 reintentos contra el mismo DataNode, a los 1, 2 y 4 s. Si sigue sin responder, el siguiente destino del plan pasa a ser el primero **[N7]**
Fallo al bajar un bloque, o firma incorrecta	Se pasa de inmediato a la siguiente réplica
Error del servidor, como `409` o `404`	No se reintenta: es una decisión, no un fallo pasajero
La reserva venció antes de confirmar (`410`)	El cliente pide un plan nuevo y repite la subida una vez

Cómo funciona una subida



1. El cliente calcula el SHA-256 del archivo y pide un plan con la ruta, el tamaño y la firma.
2. El líder valida la ruta, comprueba que nadie más la esté escribiendo y que el archivo quepa, calcula tamaño de bloque, número de copias y destinos, y reserva la ruta por Raft. El archivo todavía no existe para nadie.
3. El cliente lee cada bloque del disco, calcula su firma y lo envía al primer destino, hasta cuatro a la vez. Mientras sube, renueva la reserva cada 20 s.
4. El primer DataNode verifica la firma, cifra el bloque, lo guarda y lo propaga al siguiente destino, para que el cliente envíe cada bloque una sola vez (Hito 1, N12).
5. El cliente confirma. Antes de publicar, el líder le pregunta a cada DataNode del plan si tiene el bloque íntegro y si fue esta cuenta quien lo subió. Si todo está, publica el manifiesto del archivo por Raft, y desde ese instante el archivo existe completo.

Cómo funciona una descarga



Cada bloque verificado se escribe en su posición de un archivo temporal. Al terminar, el cliente compara el SHA-256 del archivo completo con el del manifiesto y solo entonces le da el nombre final. Una descarga fallida nunca deja un archivo que parezca completo.

---

5. Algoritmos de particionamiento y distribución

Las decisiones de esta sección viven en `controlnode/planner.go` como funciones puras y tienen pruebas automáticas.

Tamaño de bloque

Se mantiene la fórmula del Hito 1 (N3):

```
B = clamp( block_size_min , S / (k · N) , block_size_max )
```

donde `S` es el tamaño del archivo, `N` el número de DataNodes vivos y `k` los bloques mínimos por nodo, redondeando a la potencia de dos inferior. Con los valores por defecto (8 MiB, 64 MiB y `k = 4`):

Archivo	Bloque con 3 nodos	Bloques	Bloque con 4 nodos	Bloques
1 MB	8 MiB	1	8 MiB	1
100 MiB	8 MiB	13	8 MiB	13
500 MB	32 MiB	15	16 MiB	30
1 GiB	64 MiB	16	64 MiB	16

Archivo	Bloque con 3 nodos	Bloques	Bloque con 4 nodos	Bloques
10 GiB	64 MiB	160	64 MiB	160

Un archivo vacío ocupa un bloque de cero bytes, para que también tenga manifiesto.

Número de copias

Se aplican las dos reglas del Hito 1 (N4): el factor configurado, 2 por defecto, nunca supera `N - 1` con `N` el número de DataNodes registrados, y si hay menos DataNodes vivos que ese factor se guardan las copias posibles en vez de rechazar la escritura (Hito 2, N2). Las copias de un mismo bloque siempre quedan en DataNodes distintos.

Ubicación de los bloques

Se usa la asignación voraz al nodo menos cargado, el *list scheduling* de Graham (Hito 1, N5): cada copia de cada bloque va al DataNode vivo con menor porcentaje de ocupación que no tenga ya una copia de ese bloque. Sobre eso, el algoritmo implementado hace cuatro cosas:

- **Carga virtual.** Mientras arma el plan, le suma a cada nodo elegido el tamaño del bloque, para que los bloques de una misma subida no caigan todos en los nodos que estaban más vacíos al empezar (Hito 2, N3).
- **Reservas en curso.** Antes de empezar, le suma a cada nodo lo que otras subidas todavía en curso ya tienen asignado allí, porque esos bytes aún no aparecen en su disco.
- **Umbral de ocupación.** Un nodo no recibe un bloque si con él pasaría de `FULL_THRESHOLD_PERCENT`, 90 % por defecto. Si algún bloque no consigue todas sus copias, el plan se rechaza con `507` antes de subir nada.
- **Desempate por identificador**, para que el mismo estado produzca siempre el mismo plan.

En el clúster de prueba, un archivo de 40 MiB produjo 6 bloques y 12 copias, repartidas exactamente 4, 4 y 4 entre los tres DataNodes, sin ningún bloque con sus dos copias en el mismo nodo.

Identificación por contenido

El identificador de cada bloque es el SHA-256 de su contenido. Eso da tres propiedades sin código adicional: un bloque recibido se puede verificar contra su propio nombre, subir dos veces el mismo bloque no lo duplica, y dos archivos con un bloque idéntico comparten su almacenamiento.

Rebalanceo

Cuando entra un DataNode vacío, las escrituras nuevas lo prefieren de inmediato, porque es el menos ocupado. Si además la diferencia de ocupación entre el nodo más lleno y el más vacío supera el umbral de rebalanceo, 10 % por defecto, el líder mueve bloques ya guardados, uno por ciclo de mantenimiento, como previó el Hito 1. Cómo se mueve un bloque sin poner en riesgo los datos se explica en la sección 7.

6. Alta disponibilidad

ControlNodes

Los tres ControlNodes forman un grupo de Raft. Uno es el líder y los otros dos son seguidores que reciben y guardan cada cambio. El líder envía un latido a los seguidores; si un seguidor deja de recibirlo por 2 s, se postula y pide votos, y gana si consigue la mayoría.

- **Cae el líder.** En 2 a 4 s los dos que quedan eligen un líder nuevo. Como todo cambio confirmado estaba en al menos dos de los tres registros, y Raft solo deja ganar a un candidato cuyo registro esté al día, el líder nuevo tiene todos los cambios confirmados. El cliente detecta la caída, recorre la lista de ControlNodes y encuentra al nuevo líder.
- **Cae un seguidor.** El líder sigue teniendo mayoría con el otro y no se nota nada.
- **Caen dos.** El que queda no tiene mayoría y rechaza todas las operaciones con `503`. Es lo correcto: aceptar escrituras sin mayoría permitiría que, al reconectarse la red, existieran dos versiones distintas de los metadatos [**N3**].
- **Vuelve un nodo caído.** Recupera su registro y sus snapshots del disco y el líder le envía lo que le falte.

El líder atiende también las lecturas, como `ls` o `stat`. Antes de responder comprueba que sigue siendo líder con mayoría y que ya aplicó todo lo confirmado, para no devolver datos viejos [**N4**].

DataNodes

Cada DataNode envía un heartbeat cada 3 s, y tras tres perdidos, 9 s, se le da por caído (Hito 1, N9). A partir de ahí:

- **Lecturas.** El plan de lectura trae todas las réplicas vivas de cada bloque; si una no responde, el cliente pasa a la siguiente. Una descarga no falla mientras cada bloque tenga una copia viva.
- **Escrituras.** Los planes nuevos ya no incluyen al nodo caído. Si cae durante una subida, el cliente usa el siguiente destino del plan y el archivo se publica con las copias que sí quedaron, como definió el Hito 1.
- **Reparación.** Cada 5 s el líder pide el inventario real a cada DataNode vivo y compara, bloque por bloque, cuántas copias válidas hay contra cuántas deberían haber. Si falta una, le ordena a un DataNode que tiene el bloque que lo copie al nodo menos ocupado que no lo tenga, verifica la copia y actualiza el mapa por Raft. Repara un bloque por ciclo para no saturar la red.

---

7. Replicación y consistencia

Datos: bloques inmutables

Un bloque guardado no se modifica nunca y su nombre es la firma de su contenido. Dos copias de un mismo bloque son idénticas por construcción, así que no hay nada que reconciliar entre réplicas. Cambiar un archivo no cambia sus bloques: produce bloques nuevos y un manifiesto nuevo.

Metadatos: un registro replicado

Cada operación que cambia metadatos sigue el mismo camino: el líder toma una copia del estado, aplica la operación sobre ella y la propone a Raft. Cuando una mayoría la guardó en disco, los tres ControlNodes la aplican en el mismo orden. Cada entrada lleva el estado completo con un número de revisión, y la máquina de estados solo acepta la revisión siguiente a la que tiene, así que un estado viejo nunca puede pisar uno más nuevo [**N2**].

Un archivo aparece completo o no aparece

- **Un solo escritor.** Pedir un plan reserva la ruta. Otro plan, un `lock`, un `rm` o un `mv` sobre esa ruta reciben `409` mientras la reserva esté vigente. La reserva dura 60 s, el cliente la renueva mientras sube, y si el cliente muere vence sola.
- **Publicación atómica.** El archivo solo existe cuando el commit se confirma por Raft, y el commit solo se acepta después de verificar en los DataNodes que los bloques están. Repetir el commit con el mismo identificador es seguro.
- **Lectores con versión fija.** Un `open` de lectura guarda la versión del archivo en ese momento. Si otro la reemplaza, el lector sigue leyendo la suya hasta cerrar.
- **Escritura parcial con copia.** `write` baja la versión actual, aplica los bytes nuevos en la posición indicada y publica el resultado como versión nueva [**N5**].

Mantenimiento sin riesgo para los datos

Mover o borrar copias es donde un error puede perder datos, así que el mantenimiento sigue siempre el mismo orden: primero copia, luego verifica la copia en el destino, luego publica el mapa nuevo por Raft, y solo después borra. Una copia que deja de ser necesaria, por rebalanceo o porque sobra, queda *retirada*: sigue en disco al menos 120 s, más que cualquier permiso de lectura ya emitido, y antes de borrarla se comprueba otra vez que el bloque tenga suficientes copias en otros nodos. Los bloques que ningún archivo usa, como los de un archivo borrado o una subida abortada, se borran con la misma espera, y nunca mientras haya una subida en curso, porque sus bloques todavía no aparecen en ningún manifiesto.

Garantías

Situación	Garantía
Al escribir	Un solo escritor por archivo, y el archivo aparece completo o no aparece
Al replicar	Las copias de un bloque son idénticas y se verifican antes de contarlas
Al leer	Toda lectura exitosa devuelve exactamente lo publicado, verificado bloque a bloque y como archivo completo
Si cae un DataNode	Las copias restantes sirven sin reconciliación y el líder repone las faltantes
Si cae un ControlNode	Ningún cambio confirmado se pierde; con mayoría el servicio sigue
Si se reinicia todo	Metadatos y bloques se recuperan del disco de cada nodo

8. Seguridad

En la red

Todas las APIs usan HTTPS con TLS 1.2 o superior y un certificado emitido por una CA propia del clúster, que el cliente y los nodos verifican. Raft exige además TLS mutuo: cada ControlNode presenta el certificado del clúster y exige el del otro, así que una máquina ajena no puede unirse ni votar. Las peticiones internas entre nodos llevan la clave del clúster en `X-Cluster-Key`.

En disco

Cada bloque se guarda cifrado con AES-256-GCM, con un *nonce* aleatorio por copia y el identificador del bloque como dato autenticado. GCM cifra y además detecta cualquier modificación: un bloque alterado en disco no se descifra, el DataNode lo reporta como corrupto, el cliente lo lee de otra réplica y el líder repone la copia. Al leer, el DataNode además recalcula el SHA-256 del bloque descifrado.

Usuarios y permisos

- **Autenticación.** Usuario y contraseña. El ControlNode guarda solo el hash bcrypt de cada contraseña. El login entrega un token de sesión firmado con HMAC-SHA256 que dura una hora.
- **Árbol privado.** Cada ruta se guarda asociada a su dueño, y el servidor decide qué ve cada usuario: nadie puede listar, leer ni modificar el árbol de otro.
- **Permisos por bloque.** El token de sesión no sirve en los DataNodes. Para cada acceso, el líder firma un permiso que dice qué se puede hacer (leer o escribir), en qué DataNode, sobre qué bloque, hasta cuándo y con qué tamaño máximo. El DataNode solo verifica la firma, sin consultar al ControlNode [**N6**].
- **Autoría de los bloques.** Conocer el SHA-256 de un bloque ajeno no da acceso a él. El DataNode anota qué cuenta subió cada bloque, y el líder solo publica un archivo si los DataNodes confirman que fue esa cuenta quien subió sus bloques. Sin esto, alguien podría "publicar" un archivo con hashes ajenos y luego descargarlo.

Secretos

`scripts/setup.py` genera la CA, el certificado, la clave del clúster, la clave de firma, la clave de cifrado y el primer usuario con contraseña aleatoria. Quedan en una carpeta que no se sube a Git y que los contenedores montan como solo lectura.

---

9. Entorno de ejecución

Cada nodo corre en un contenedor de Docker o de forma nativa, y expone su propia API. Se usaron tres entornos.

Docker Compose en una máquina

Servicio	Qué es	Puerto
`cn1`, `cn2`, `cn3`	ControlNodes, cada uno con su volumen de metadatos	8000 y 9000
`dn1`, `dn2`, `dn3`	DataNodes, cada uno con su volumen de bloques	8001
`dn4`	Cuarto DataNode, apagado por defecto (perfil `scale`)	8001
`client`	Cliente, como herramienta (perfil `tools`)	ninguno

```
python3 scripts/setup.py
docker compose up -d --build cn1 cn2 cn3 dn1 dn2 dn3
python3 scripts/demo.py
```

Procesos nativos

La prueba de integración compila los binarios y levanta 3 ControlNodes y 4 DataNodes como procesos independientes con TLS real, cada uno con su puerto y su carpeta de datos. No necesita Docker.

Varias máquinas en AWS Academy

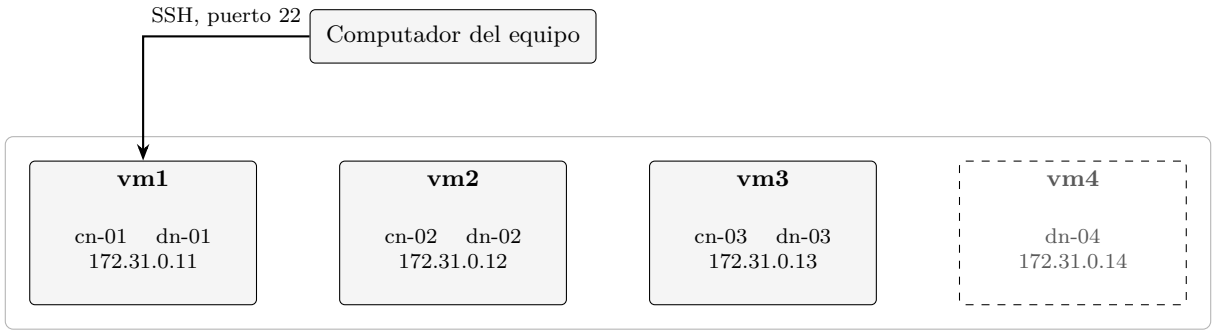


Figura 2: Despliegue en AWS. El cliente corre en la vm1.

Tres instancias EC2 con Ubuntu 24.04, cada una con un ControlNode y un DataNode, y una cuarta opcional solo con un DataNode para mostrar el escalado. Todas usan el mismo archivo de Docker Compose (`deploy/aws/compose.yml`) con red del host, y cada una tiene un archivo `.env` que dice su identidad y su IP privada. Los nodos se hablan por las IP privadas de la VPC, y el grupo de seguridad solo abre los puertos del clúster entre sus miembros; desde Internet solo se abre SSH, y el cliente corre en la vm1 **[N8]**. El paso a paso está en `deploy/aws/README.md`.

Antes de usar AWS, ese mismo archivo de Compose se probó en una máquina simulando cuatro VMs, cada una como un proyecto de Docker distinto con su propia IP de *loopback* y su propio disco (sección 10).

Parámetros

Todos los parámetros se leen de variables de entorno al arrancar, así que se cambian sin recompilar. Un valor inválido detiene el arranque con un mensaje que dice qué variable está mal.

Variable	Defecto	Qué controla
`REPLICATION_FACTOR`	2	Copias de cada bloque
`BLOCK_SIZE_MIN_MB`, `_MAX_MB`	8, 64	Rango del tamaño de bloque
`BLOCKS_PER_NODE`	4	El `k` de la fórmula
`HEARTBEAT_MS`, `HEARTBEAT_MISSES`	3000, 3	Un DataNode se da por caído a los 9 s
`REPORT_MS`	30000	Reporte de bloques de cada DataNode
`WRITE_LEASE_SECONDS`	60	Duración de la reserva de escritura
`MAINTENANCE_MS`	5000	Ciclo de reparación y rebalanceo
`GC_GRACE_SECONDS`	120	Espera antes de borrar una copia
`FULL_THRESHOLD_PERCENT`	90	Ocupación máxima de un DataNode
`REBALANCE_THRESHOLD_PERCENT`	10	Diferencia que dispara el rebalanceo
`CAPACITY_BYTES`	1 GiB	Espacio que ofrece cada DataNode

---

10. Pruebas y análisis de resultados

Pruebas automáticas

`python3 scripts/test.py` corre todo: 32 pruebas unitarias en Go, 32 en Python, `go vet` y la prueba de integración. El repositorio tiene además integración continua en GitHub que corre lo mismo en cada cambio.

Qué se prueba	Casos
Planificador	Tamaño de bloque, número de bloques y copias en los escenarios de la sección 5, copias en nodos distintos, reparto parejo, preferencia por el nodo más vacío y umbral de ocupación
ControlNode	Reserva y vencimiento, commit que no confía en el cliente, publicación con las copias disponibles, árboles separados por usuario, un solo escritor, lector con versión fija, estado viejo rechazado por la máquina de estados, snapshot y restauración
DataNode	Cifrado y escritura atómica con subidas simultáneas del mismo bloque, permisos por nodo, bloque y vencimiento, rangos, capacidad, propagación, inventario que vuelve a verificar un bloque alterado, autoría
Cliente	Reintentos y su espera, cambio de destino si el primero no responde, búsqueda del líder, renovación de la reserva, lectura con otra réplica, descarga que verifica antes de reemplazar, `write` con copia

Prueba de integración

`scripts/integration_test.py` levanta el clúster con procesos reales y TLS y recorre estos escenarios. Todos pasaron en la última ejecución: los quince primeros tardan unos 25 s y los dos lentos, que esperan los plazos reales de 60 y 120 s, unos 4 minutos.

Escenario	Resultado
Autenticación y árbol privado	Contraseña incorrecta: `401`. Cada usuario ve solo su árbol
Archivo de 24 MiB	3 bloques, 2 copias cada uno, 2 copias por DataNode, SHA-256 idéntico al bajarlo
Archivo vacío	Un bloque de 0 bytes con 2 copias; se recupera vacío
Subida abortada o incompleta	Nunca se publica. Un commit con menos bloques que el plan: `409`
Permisos de bloques	El token de sesión, un permiso vencido o uno de otro nodo: `401` en el DataNode
Hash de un bloque ajeno	Publicar un archivo con bloques que subió otra cuenta: `409`
Rango, handles, lock y write	Un rango que cruza el límite entre dos bloques es correcto; segundo escritor: `409`; el lector conserva su versión; `write` cambia solo los bytes indicados
Bloque corrupto en disco	El DataNode responde `500`, el cliente usa la otra réplica y el líder repone la copia
Cae un DataNode	La descarga usa otras réplicas y el líder deja todos los bloques otra vez con 2 copias
Cae el líder	Hay nuevo líder y el cliente lo encuentra solo; los metadatos confirmados siguen ahí y se aceptan escrituras nuevas
Cuarto DataNode	Se registra en caliente y recibe bloques por rebalanceo
Reinicio de todo el clúster	Archivos y directorios intactos, SHA-256 verificado
Solo un ControlNode vivo	Rechaza la escritura con `503` y al volver la mayoría la escritura no existe
Renovación y vencimiento	Una subida lenta renueva su reserva; una reserva abandonada vence a los 60 s
Borrado de huérfanos	Tras 120 s se borran los bloques sin uso y las copias sobrantes, y el archivo sigue íntegro

Pruebas con Docker y con cuatro VMs simuladas

Además se recorrió a mano el uso real: con Docker Compose y con el archivo de despliegue de AWS simulando cuatro VMs en una máquina, con el cliente nativo como se usará en AWS.

Prueba	Resultado
Subir 40 MiB (Docker)	6 bloques, copias repartidas 4, 4 y 4 entre los DataNodes
Apagar `dn1` y bajar el archivo	SHA-256 idéntico. En 25 s el líder repuso las copias que estaban en `dn1`
Apagar el ControlNode líder	Respuesta del nuevo líder en 4 s, con `replicas_ok` correcto
Agregar `dn4`	Recibe de inmediato las escrituras nuevas; con el umbral en 2 % recibe bloques viejos por rebalanceo, uno cada 5 s
Apagar la VM del líder completa y subir 100 MiB en ese momento	La subida termina en 17 s usando los destinos vivos; el archivo queda publicado y en 75 s todos sus bloques vuelven a tener 2 copias
Agregar la vm4 y reiniciar las cuatro VMs	Todos los archivos intactos y con 2 copias

Rendimiento

Sobre las cuatro VMs simuladas se subió y bajó un archivo de 200 MiB (13 bloques de 16 MiB, 2 copias) con 1 y con 4 workers, tres veces cada uno, verificando el SHA-256 en cada descarga. La tabla muestra la mediana.

Workers	Subida	Caudal de subida	Bajada	Caudal de bajada
1	3.03 s	66 MiB/s	1.16 s	172 MiB/s
4	1.73 s	115 MiB/s	0.60 s	332 MiB/s

Con 4 workers la subida es 1.75 veces más rápida y la bajada 1.93 veces. El paralelismo por bloques funciona: mientras un bloque viaja, otro se lee del disco y se firma, y los DataNodes atienden a la vez. La subida es más lenta que la bajada porque cada bloque se escribe dos veces, una en cada copia, se cifra y se sincroniza a disco antes de confirmar. Los números absolutos no representan una red real, porque las VMs simuladas comparten procesador y disco y el tráfico no sale de la máquina [**N10**].

Resultados en AWS

Esta subsección se completa con el despliegue en AWS Academy, repitiendo sobre las VMs reales las pruebas de la tabla anterior y la medición de rendimiento.

Prueba	Resultado
Subir y bajar 100 MiB	Pendiente
Apagar un DataNode	Pendiente
Apagar la VM del líder	Pendiente
Agregar la vm4	Pendiente
Rendimiento con 1 y 4 workers	Pendiente

Hallazgos corregidos durante las pruebas

Las pruebas en Docker y en las VMs simuladas encontraron cinco problemas que la prueba de integración no veía, porque ahí todo corre en una máquina y una caída responde al instante. Todos se corrigieron y tienen prueba automática cuando fue posible:

- **Un líder nuevo no conocía a los DataNodes.** Solo el líder aceptaba heartbeats, así que justo después de una elección creía que no había DataNodes vivos y por unos segundos respondía `replicas_ok: 0` y fallaba descargas. Ahora los seguidores también los reciben.
- **El cliente se rendía durante la elección.** Esperaba 3 s en total y una elección puede tardar 4. Ahora espera 1, 2 y 4 s.
- **Una subida justo después de una caída fallaba.** Durante los 9 s en que el nodo caído todavía parece vivo, el plan lo incluye. Ahora el cliente usa el siguiente destino y el líder publica con las copias que sí quedaron, como definió el Hito 1.
- **El inventario descifraba todo el disco en cada heartbeat.** Cada 3 s el DataNode volvía a verificar todos sus bloques mientras bloqueaba las lecturas. Ahora cada bloque se verifica una vez y de nuevo solo si su archivo cambia, y las lecturas no esperan a nadie.

- **Un nodo apagado podía congelar al líder.** En AWS una máquina apagada no rechaza la conexión, simplemente no contesta. Las consultas cortas del líder a los DataNodes ahora tienen un límite de 10 s y cualquier conexión entre nodos uno de 5 s.

---

11. Cambios respecto a los hitos anteriores

El Hito 1 previó que los ajustes se documentarían en los hitos siguientes (RNF8). Estos son los cambios de decisión:

1. Raft en lugar del algoritmo del matón modificado. Ver [**N1**].

2. Las lecturas las atiende el líder, no las réplicas. El Hito 1 proponía que las réplicas atendieran consultas de solo lectura. Una réplica puede ir atrasada y devolver, por ejemplo, un archivo recién borrado [**N4**].

3. `cd` y las primitivas de RF3 se exponen como comandos. El Hito 2 los había dejado como modelo interno. Ahora el cliente guarda una sesión con el directorio actual y los handles abiertos, y el ControlNode tiene endpoints para `open`, `lock`, `close` y la lectura con versión fija.

4. Sin permisos compartidos. El Hito 1 mencionaba conceder permiso sobre un directorio a otro usuario. Se dejó fuera [**N9**].

5. El commit verifica en los DataNodes. El Hito 2 confiaba en la lista de copias que reportaba el cliente. Ahora el líder lo comprueba en cada DataNode antes de publicar.

6. `X-Forward-To` lleva los permisos. Como cada escritura en un DataNode exige un permiso firmado para ese nodo, la cabecera pasó a ser una lista JSON con la URL, el nodo y el permiso de cada destino siguiente.

---

12. Limitaciones

- **Tres ControlNodes fijos.** Agregar un ControlNode exige reconfigurar Raft, y no se implementó. Con tres se tolera la caída de uno.
- **Estado completo en cada entrada.** Simplifica mucho la consistencia, pero el costo de cada cambio crece con la cantidad de metadatos [**N2**].
- **Una copia durante una falla.** Un bloque escrito mientras un DataNode está caído puede quedar con una sola copia hasta que la reparación lo complete; si en ese intervalo cae también el nodo que la tiene, el bloque no está disponible hasta que vuelva.
- **99,5 % es un objetivo.** El Hito 1 derivó el factor de replicación de un objetivo de disponibilidad. Las pruebas demuestran que el sistema sobrevive a las fallas, pero no miden una disponibilidad anual.
- **Confianza entre nodos.** Todos comparten las mismas claves del clúster; quien controle un nodo controla el clúster. No hay rotación de la clave de cifrado.

Notas

[N1] Por qué Raft y no el algoritmo del matón. El Hito 1 propuso el algoritmo del matón con dos cambios: gana quien tenga los metadatos más recientes y se exige mayoría. Al implementarlo apareció lo que le faltaba. El matón elige un líder, pero no dice qué hacer con los cambios: cuándo un cambio cuenta como confirmado, qué pasa con uno que el líder anterior alcanzó a mandar a un solo seguidor antes de caerse, o cómo un nodo que vuelve descarta lo que el resto nunca aceptó. Resolver eso bien es justamente lo que hace Raft, con términos numerados, votos registrados en disco y la regla de que un cambio está confirmado cuando una mayoría lo guardó. Se usa la implementación de HashiCorp, la misma de Consul y Nomad, en vez de escribir una propia, porque un error sutil en consenso pierde datos y no se nota en pruebas cortas. Lo que se conserva del Hito 1 es la idea: tres nodos simétricos, mayoría para decidir y el líder con los datos más recientes.

[N2] Por qué cada entrada lleva el estado completo. La alternativa es que cada entrada describa una operación, como "crear el directorio X", y que cada nodo la aplique. Eso obliga a que aplicar una operación sea determinista en los tres nodos: misma hora, mismos identificadores aleatorios, mismo orden de recorrido de mapas. Si una operación se aplica distinto en un nodo, los metadatos divergen sin que nadie lo note. Con el estado completo, el líder decide una sola vez y los seguidores copian el resultado, y el número de revisión impide que un estado viejo reemplace uno nuevo. El costo es que cada cambio envía todos los metadatos. Con los volúmenes del proyecto, decenas de archivos y unos cientos de bloques, son pocos kilobytes por cambio. Para millones de archivos habría que pasar a operaciones.

[N3] Por qué sin mayoría no se atiende. Si un ControlNode aislado aceptara escrituras, al reconectarse la red habría dos historias distintas de los metadatos y una de ellas se perdería. Entre seguir funcionando con riesgo de perder datos confirmados o dejar de responder, el sistema escoge dejar de responder. Es la decisión que toma cualquier sistema basado en consenso.

[N4] Por qué el líder comprueba antes de leer. Un líder que quedó aislado puede creer que todavía es líder por un momento, mientras los otros dos ya eligieron a otro y aceptaron cambios. Si respondiera lecturas con lo que tiene, podría mostrar un archivo ya borrado. Por eso, antes de cada operación, el líder confirma con la mayoría que sigue siéndolo y espera a tener aplicado todo lo confirmado.

[N5] Por qué `write` publica una versión nueva. Modificar un bloque en su lugar rompería la propiedad que sostiene todo el diseño: que dos copias de un bloque son idénticas porque nunca cambian. Además, un lector que estuviera leyendo ese bloque vería una mezcla de las dos versiones. Con la copia, el archivo pasa de una versión completa a otra completa en un solo commit. El costo es que escribir pocos bytes en un archivo grande transfiere el archivo entero; para archivos de escritura única, que es el caso del proyecto, no es un problema.

[N6] Por qué permisos firmados y no consultas al ControlNode. Si cada acceso a un bloque exigiera preguntarle al ControlNode si está permitido, todo el tráfico de datos volvería a pasar por él y la separación de planos perdería sentido. Con un permiso firmado, el ControlNode decide una vez al armar el plan, y el DataNode solo verifica la firma con la clave compartida. El permiso está atado a un nodo, un bloque, una operación y un plazo corto, así que si se filtra sirve para muy poco.

[N7] Por qué cambiar de destino y no rechazar la subida. El Hito 1 definió que, con nodos caídos, se guardan las copias posibles y se completan las faltantes después, porque rechazar la escritura dejaría el servicio inutilizable justo cuando más se necesita. La reparación automática hace que el estado degradado dure solo hasta el siguiente ciclo de mantenimiento.

[N8] Por qué el cliente corre dentro de la VPC. El cliente habla directamente con los DataNodes, así que necesita alcanzar sus direcciones. Dentro de la VPC basta con las IP privadas y no hay que abrir ningún puerto del clúster a Internet. Desde afuera habría que anunciar las IP públicas, que en el Learner Lab cambian cada vez que se inicia el laboratorio.

[N9] Por qué no hay permisos compartidos. El enunciado pide control de acceso por usuario o por grupo, y el árbol privado por usuario lo cumple de forma completa y verificable. Compartir directorios agrega listas de permisos que se heredan, se revocan y se mueven con `mv`, con sus propios casos de borde, sin aportar

a ninguno de los criterios de alta disponibilidad, replicación o consistencia, que es donde estaba el riesgo del proyecto.

[N10] Por qué la medición es solo indicativa. En una sola máquina las transferencias no cruzan una red real y compiten por el mismo procesador y el mismo disco. Sirve para confirmar que el paralelismo reduce el tiempo, pero no para dimensionar el sistema; para eso está la medición en AWS.

Referencias

- S. Ghemawat, H. Gobioff y S.-T. Leung. *The Google File System*. ACM Symposium on Operating Systems Principles, 2003.
- K. Shvachko, H. Kuang, S. Radia y R. Chansler. *The Hadoop Distributed File System*. IEEE Symposium on Mass Storage Systems and Technologies, 2010.
- D. Ongaro y J. Ousterhout. *In Search of an Understandable Consensus Algorithm*. USENIX Annual Technical Conference, 2014.
- R. L. Graham. *Bounds for certain multiprocessing anomalies*. Bell System Technical Journal, 45(9), 1966.
- M. Dworkin. *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. NIST SP 800-38D, 2007.
- HashiCorp. *Raft*, biblioteca en Go. `https://github.com/hashicorp/raft`.
- A. Vélez Rendón y J. L. Restrepo. *DFSha: Especificación del proyecto*, Hito 1, 2026. En `docs/hito-1`.
- A. Vélez Rendón y J. L. Restrepo. *DFSha: Arquitectura distribuida y comunicaciones*, Hito 2, 2026. En `docs/hito-2`.